

DOCUMENTACIÓN DEL PROYECTO

Sistema de Inventario

Aplicación de escritorio en Java Swing con SQLite

Presentan

Hernández Cruz Karla Sofia

Olvera Hernández Estephany

Higinio Reséndiz

Sergio Abimael

García Falcon Luis Raúl

Grupo: _____

Fecha: _____

1. Descripción general

Sistema de Inventario es una aplicación de escritorio diseñada para administrar categorías y productos. Su objetivo es centralizar el registro de existencias, precios y clasificación de artículos mediante una interfaz gráfica sencilla. La solución incorpora programación orientada a objetos, persistencia en una base de datos relacional SQLite, operaciones CRUD y manejo controlado de errores.

Funciones principales

- Alta, consulta, modificación y eliminación de categorías.
- Alta, consulta, modificación y eliminación de productos.
- Búsqueda de productos por nombre.
- Validación de campos obligatorios, precios y existencias no negativas.
- Restricción de integridad referencial: una categoría con productos no puede eliminarse.

2. Arquitectura orientada a objetos

El proyecto se organiza por responsabilidades. Esta separación favorece el encapsulamiento, la reutilización y el mantenimiento del código.

Paquete	Responsabilidad
model	Entidades Categoria y Producto. Sus atributos son privados y se consultan o modifican mediante getters y setters.
dao	Acceso a datos: consultas SQL parametrizadas y operaciones CRUD.
db	Conexión reutilizable e inicialización de las tablas SQLite.
service	Reglas de negocio y validaciones antes de persistir datos.
ui	Ventana principal y diálogo de categorías construidos con Java Swing.
exception	Excepciones específicas DataAccessException y ValidationException.

3. Persistencia y modelo relacional

La base de datos utiliza SQLite, por lo que se genera como un archivo local y no requiere un servidor externo. La clase DatabaseConnection concentra la URL JDBC, obtiene conexiones y prepara el esquema al iniciar la aplicación. Los DAO usan PreparedStatement, lo que evita concatenar entradas del usuario en las sentencias SQL.

Modelo de datos

```
CATEGORIAS PRODUCTOS
-----
id (PK) id (PK)
nombre (UNIQUE, NOT NULL) nombre (NOT NULL)
descripcion precio (CHECK >= 0)
existencia (CHECK >= 0)
categoria_id (FK -> categorias.id)
```

La relación es uno a muchos: una categoría puede clasificar varios productos, pero cada producto debe pertenecer a una sola categoría. La llave foránea y la política ON DELETE RESTRICT conservan la integridad de la información.

Script de base de datos

El archivo database/KSEHSL_GRUPO.sql crea ambas tablas y carga tres categorías y tres productos de prueba. Puede ejecutarse con SQLiteStudio, DB Browser for SQLite o la consola sqlite3.

4. Manejo de excepciones

Las operaciones críticas se protegen con try-with-resources, que cierra automáticamente conexiones, sentencias y resultados. Los errores de SQL se encapsulan en DataAccessException con mensajes comprensibles; las reglas de negocio se comunican mediante ValidationException. La interfaz captura estas excepciones y muestra un cuadro de diálogo, en lugar de finalizar la aplicación inesperadamente.

5. Interfaz gráfica y uso

La ventana principal contiene un formulario de producto, un selector de categoría, una tabla de resultados y controles para guardar, actualizar, eliminar, limpiar y filtrar. El botón Gestionar categorías abre una segunda ventana para administrar el catálogo de categorías.

Proceso de uso

1. Ejecute el proyecto y cree primero al menos una categoría.
2. Capture nombre, precio, existencia y categoría del producto.
3. Presione Guardar para registrar el artículo.
4. Seleccione una fila de la tabla para cargar sus datos y posteriormente Actualizar o Eliminar.
5. Escriba un texto en Buscar y presione Filtrar para consultar por nombre.

6. Instalación y ejecución

El proyecto incluye un archivo Maven que declara el controlador JDBC de SQLite. Se requiere JDK 21 o superior y Maven 3.9 o superior. Desde la carpeta raíz del proyecto se ejecuta:

```
mvn clean compile exec:java
```

En el primer inicio se crea automáticamente la carpeta data y el archivo `inventario.db`. Si se desea ver datos de ejemplo desde el inicio, se puede ejecutar previamente el script SQL incluido.

7. Control de versiones

La entrega se preparó de forma local y no se configuró un repositorio remoto. Para completar el requisito de Git, el equipo debe inicializar el repositorio, realizar commits por integrante y después vincularlo a su propia cuenta de GitHub o GitLab. Se recomienda utilizar mensajes descriptivos, por ejemplo: `git commit -m "Agrega CRUD de productos"`.

8. Verificación

La totalidad de las clases Java fue compilada con JDK 24 en modo de compatibilidad Java 21. Antes de entregar, se recomienda ejecutar el comando Maven indicado en un equipo con acceso a Maven Central, verificar los CRUD y tomar capturas propias de la aplicación y del historial Git para la evidencia final.